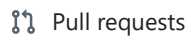
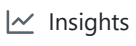


 djoseph-png / py-lockdown-cafe



 Code  Pull requests  Actions  Projects  Wiki  Security  Insights  Settings



☆ 0 stars  1.6k forks  0 watching  Branches  Activity
 Tags

🌐 Public repository · Forked from [mate-academy/py-lockdown-cafe](#)

  1 Branch  0 Tags  





This branch is up to date with [mate-academy/py-lockdown-cafe:master](#) .

 **Contribute** ▾

 **Sync fork** ▾

 **sergii-nosachenko** Merge pull request [mate-academy#1227](#) from viktor-rybenchuk/updat... 

afafeb1 · 11 months ago 

 .github/workflows	Update test.yml	3 years ago
 app	Add task	3 years ago
 tests	error messages added	3 years ago
 .flake8	Update .flake8	3 years ago
 .gitignore	Initial commit	3 years ago
 README.md	checklist added	3 years ago
 checklist.md	Merge pull request mate-academy...	11 months ago
 requirements.txt	Update requirements.txt	3 years ago

 **README**



Lockdown cafe

- Read [the guideline](#) before starting.
- If you need additional information about `datetime` module you can find it [here](#).

Implement a `Cafe` class, whose instances have one attribute `name` . Write a `visit_cafe` method with one parameter, `visitor` of type `dict` . Due to the pandemic, not everyone can visit the cafe.

If the visitor does not have a `vaccine` key, it means that he is not vaccinated. In this case, the method must raise a `NotVaccinatedError` exception.

```
kfc = Cafe("KFC")
visitor = {
```



```

        "name": "Paul",
        "age": 23,
    }
    kfc.visit_cafe(visitor)    # NotVaccinatedError

```

The vaccine must not be expired, otherwise the method should raise an `OutdatedVaccineError` exception. You can get an `expiration_date` from `visitor["vaccine"]` dictionary.

```

import datetime

kfc = Cafe("KFC")
visitor = {
    "name": "Paul",
    "age": 23,
    "vaccine": {
        "expiration_date": datetime.date(year=2019, month=2, day=23)
    }
}
kfc.visit_cafe(visitor)    # OutdatedVaccineError

```

And the last rule: all visitors must wear masks. Otherwise `visit_cafe` should raise an `NotWearingMaskError` exception.

```

kfc = Cafe("KFC")
visitor = {
    "name": "Paul",
    "age": 23,
    "vaccine": {
        "expiration_date": datetime.date.today()
    },
    "wearing_a_mask": False
}
kfc.visit_cafe(visitor)    # NotWearingMaskError

```

If all the rules are met, then the person can visit the cafe and the method should return the string `"Welcome to {cafe.name}"`

```

kfc = Cafe("KFC")
visitor = {
    "name": "Paul",
    "age": 23,
    "vaccine": {
        "expiration_date": datetime.date.today()
    },
    "wearing_a_mask": True
}
kfc.visit_cafe(visitor) == "Welcome to KFC"

```

Write a function `go_to_cafe` which takes a `friends` list and a `cafe`. It should return a string `"Friends can go to {cafe.name}"` if they are all allowed to visit it.

If at least one of the friend has problems with vaccines the function should return a message `"All friends should be vaccinated"`.

If everyone is vaccinated but somebody isn't wearing a mask the function should return a message "Friends should buy {masks_to_buy} masks" where masks_to_buy is the number of friends who don't have a mask.

```
friends = [
    {
        "name": "Alisa",
        "vaccine": {
            "expiration_date": datetime.date.today()
        },
        "wearing_a_mask": True
    },
    {
        "name": "Bob",
        "vaccine": {
            "expiration_date": datetime.date.today()
        },
        "wearing_a_mask": True
    },
]
go_to_cafe(friends, Cafe("KFC")) == "Friends can go to KFC"
```

```
friends = [
    {
        "name": "Alisa",
        "vaccine": {
            "expiration_date": datetime.date.today()
        },
        "wearing_a_mask": False
    },
    {
        "name": "Bob",
        "vaccine": {
            "expiration_date": datetime.date.today()
        },
        "wearing_a_mask": False
    },
]
go_to_cafe(friends, Cafe("KFC")) == "Friends should buy 2 masks"
```

```
friends = [
    {
        "name": "Alisa",
        "wearing_a_mask": True
    },
    {
        "name": "Bob",
        "vaccine": {
            "expiration_date": datetime.date.today()
        },
        "wearing_a_mask": True
    },
]
go_to_cafe(friends, Cafe("KFC")) == "All friends should be vaccinated"
```

Use the following project structure:

[main.py](#)
[cafe.py](#)
[errors.py](#)



All errors should be defined in the `errors.py` module. The `Cafe` class should be defined in `cafe.py` module. The function should be implemented in `main.py` module.

Notes:

- Use `try/except` to handle errors in `go_to_cafe`
- Be sure to pass descriptive messages when you raise the exception in the `visit_cafe` method
- Create a `VaccineError` parent class for `NotVaccinatedError` and `OutdatedVaccineError` errors. Use it to catch both types of errors in the same `except` clause.
- You can work with dates using `datetime` module.
- You can compare two dates using `<` operator:

```
import datetime
date1 = datetime.date(2020, 3, 4)
date2 = datetime.date(2022, 1, 30)

assert date1 < date2
```



- To get the current date use `datetime.date.today()`



Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Languages

- Python 100.0%

Suggested workflows

Based on your tech stack



Pylint

Lint a Python application with pylint.

Configure



Python application



Create and test a Python application.

Configure



Django

Build and Test a Django Project

Configure

[More workflows](#)

[Dismiss suggestions](#)